`StatusMiddleware`, it's the same as if the request came from the network.

The `HostBuilder` configuration in this example is simple. Even though I've called this an integration test, you're specifically testing the `StatusMiddleware` on its own rather than in the context of a real application. I think this setup is preferable for testing custom middleware compared with the "proper" unit tests I showed in section 36.1.

Regardless of what you call it, this test relies on simple configuration for the test app. You may also want to test the middleware in the context of your real application so that the result is representative of your app's real configuration.

If you want to run integration tests based on an existing app, you don't want to have to configure the test `HostBuilder` manually, as you did in listing 36.7. Instead, you can use another helper package, Microsoft.AspNetCore.Mvc.Testing.

36.3.2 Testing your application with WebApplicationFactory

Building up a `HostBuilder` and using the Test Host package, as you did in section 36.3.1, can be useful when you want to test isolated infrastructure components, such as middleware. However, it's also common to want to test your real app, with the full middleware pipeline configured and all the required services added to DI. This gives you the most confidence that your application is going to work in production.